

Aperçu du Framework Spring

Spring facilite la création d'applications d'entreprise en Java, avec un support optionnel pour Groovy et Kotlin. Il offre la flexibilité nécessaire pour différents types d'architectures, allant des applications autonomes aux applications cloud avec serveur embarqué. Depuis Spring 6.0, Java 17+ est requis.

Spring est open source et bénéficie d'une large communauté active, garantissant une évolution pérenne basée sur des cas d'usage concrets.

Qu'entendons-nous par "Spring"

- **Spring Framework** : base de l'écosystème, offrant un conteneur IoC et un modèle de configuration via l'injection de dépendances.
- **Écosystème Spring** : inclut Spring Boot, Spring Security, Spring Data, Spring Cloud, Spring Batch, etc. Chaque projet a son propre répertoire pour le code source, le suivi des issues et les dates de sortie (spring.io/projects).

Modules principaux :

- Core Container (IoC, DI)
- Support pour messagerie, transactions, persistance, web
- Servlet-based Spring MVC et Spring WebFlux pour applications réactives

Modules et jars :

- Compatible classpath et module path (JPMS)
- Automatic-Module-Name défini dans le manifest (ex : spring-core, spring-context)

Histoire et évolution

- Lancé en 2003 pour simplifier J2EE
- Intègre soigneusement des spécifications EE :
 - Servlet API ([JSR 340](#))
 - WebSocket ([JSR 356](#))
 - Concurrency Utilities ([JSR 236](#))
 - JSON Binding ([JSR 367](#))
 - Bean Validation ([JSR 303](#))
 - JPA ([JSR 338](#))
 - JMS ([JSR 914](#))
- Supporte également Dependency Injection ([JSR 330](#)) et Common Annotations ([JSR 250](#))
- Depuis Spring 6.0, adoption des namespaces Jakarta EE 9+

Compatibilité serveur et ORM : Tomcat 10.1, Jetty 11, Hibernate ORM 6.1

Évolution DevOps et cloud : Spring Boot permet le déploiement rapide avec serveur embarqué et peu de configuration. WebFlux peut fonctionner sans API Servlet.

Philosophie de conception

- **Choix différé** : possibilité de changer le fournisseur de persistance via configuration.
- **Diversité des perspectives** : support d'un large panel de besoins applicatifs.
- **Rétrocompatibilité forte** : évolutions conçues pour limiter les changements incompatibles.
- **Design API réfléchi** : intuitives, fiables et durables.
- **Qualité du code élevée** : javadoc précise, code propre, pas de dépendances circulaires.

Retours et contributions

- **Questions et débogage** : Stack Overflow ([spring tags](#))
- **Bugs et suggestions** : GitHub Issues ([Spring Framework Issues](#))
- **Contributions** : Pull requests via GitHub ([CONTRIBUTING guide](#))

Débutez avec Spring

- **Spring Boot** : environnement rapide pour créer des applications Spring prêtes à la production.
- **Génération de projets** : [start.spring.io](#)
- **Guides pratiques** : [Spring Guides](#), ex. [RESTful Web Service](#)

Note importante : Spring Boot favorise « convention over configuration » et simplifie le démarrage des applications.